

Weeks 1-2: Java Fundamentals

Document 2 of 6 — Days 1 to 14 (Sep 4 - Sep 17)

Fourteen days from zero Java to a tested, Maven-built application written unaided. Each day gives you a named algorithm pattern, the concepts to study, a numbered drill set, and project work. **The drills in the gold panel are the part that produces retention** — do all of them, typed by hand.

Your running project across these two weeks is a **Library Management System** console application, which grows every day.

WEEK 1 — Syntax, OOP, and Exceptions

Sep 4 - Sep 10

Day 1 — Friday, Sep 4

Setup, syntax, and your first Java code

60 min Algorithms	Pattern: Warm-up (no pattern yet) Today is about getting comfortable with the LeetCode editor and writing Java that compiles. Do not worry about optimality. Solve in this order: 1. Running Sum of 1d Array (easy) 2. Richest Customer Wealth (easy) 3. Two Sum — brute force nested loops only; do NOT look up the optimal solution, you will return to it on Day 7
150 min Learn	Install JDK 21 and IntelliJ IDEA Community. What the JVM is; compile versus run; the class/main structure; primitive types (int, long, double, boolean, char) and their ranges; String basics; variable declaration and assignment; casting and integer overflow; System.out.println and printf formatting; reading input with Scanner; comments and naming conventions.
DRILLS · Java syntax, primitives, and integer arithmetic <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write a class that declares one variable of every primitive type, prints each with its value, and prints the MIN_VALUE and MAX_VALUE of int and long. 2. Deliberately cause integer overflow: set an int to Integer.MAX_VALUE, add 1, print the result, and write a comment explaining what happened. 3. Write TemperatureConverter with celsiusToFahrenheit and fahrenheitToCelsius. Print a formatted table of -40 to 100 in steps of 20 using printf with 2 decimal places. 4. Write a program that reads a name and age with Scanner and prints a sentence using both. 5. Write SecondsConverter: take 98765 seconds and print it as days, hours, minutes, seconds using integer division and modulo. 6. Predict on paper what 7/2, 7%2, 7.0/2, and (int)(7.0/2) each produce, then write code to check every one. Note any you got wrong. 7. Write a program that swaps two variables, first with a temp variable, then without one using arithmetic.	
120 min Build	Create a public GitHub repo named java-journey with a README. Commit each drill as its own file in a package called day01. Make at least three separate commits today so you start the habit properly.

✓ **Win of the day: Working Java environment, a public repo, and seven programs you wrote from a blank file.**

Day 2 — Saturday, Sep 5

Control flow, loops, and methods

60 min Algorithms	Pattern: Arrays & Hashing (part 1) — counting with a frequency map The core insight of the whole pattern: instead of scanning the array repeatedly, build a HashMap of value to count in one pass, then answer questions from the map. Solve in this order: 1. Contains Duplicate (easy) — use a HashSet 2. Valid Anagram (easy) — frequency counting 3. Majority Element (easy) — frequency map, then find the max
150 min Learn	Arithmetic, comparison, and logical operators; short-circuit evaluation; the ternary operator; if/else if/else; switch statements and switch expressions; for, enhanced for, while, do-while; break and continue; nested loops. Then methods: declaration, parameters, return types, void, overloading, static versus instance, and pass-by-value semantics.
DRILLS · Control flow, loops, and method design <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write FizzBuzz for 1 to 100, first with if/else, then rewrite it with a switch expression. 2. Write a method isPrime(int n) and use it to print all primes under 200. Count them and print the count. 3. Write printTriangle(int height) that prints a right-angled triangle of asterisks, then printPyramid(int height) that prints a centred pyramid. Nested loops, no string repetition tricks. 4. Write a multiplication table from 1 to 12 formatted in aligned columns with printf. 5. Write three overloaded versions of a method named area: area(double side) for a square, area(double w, double h) for a rectangle, area(double radius, boolean isCircle) for a circle. Call all three. 6. Write sumDigits(int n) and reverseNumber(int n) using only loops and modulo, no strings. 7. Write a method that takes an int and returns whether it is an Armstrong number, and print all Armstrong numbers under 10000. 8. Demonstrate pass-by-value: write a method that tries to reassign an int parameter and one that tries to reassign an array parameter, print the results, and write a comment explaining the difference.	
120 min Build	Build a menu-driven console calculator using Scanner and a switch, supporting add, subtract, multiply, divide, and power, looping until the user chooses exit. Handle division by zero without crashing. Then build a number-guessing game with feedback and an attempt counter.

✓ **Win of the day: Two interactive programs running, and you can write any loop or conditional from memory.**

Day 3 — Sunday, Sep 6

Arrays and Strings

60 min Algorithms	Pattern: Arrays & Hashing (part 2) — grouping and bucketing When you need items grouped by some derived key, compute the key and use it as a HashMap key whose value is a list. Bucketing by count is the same idea applied to frequencies. Solve in this order: 1. Group Anagrams (medium) — sorted string as the map key 2. Top K Frequent Elements (medium) — frequency map then bucket sort 3. Product of Array Except Self (medium) — prefix and suffix passes
150 min Learn	Array declaration, initialization, length, iteration, and default values; <code>ArrayIndexOutOfBoundsException</code> ; 2D arrays and row/column iteration; <code>Arrays.toString</code> , <code>Arrays.sort</code> , <code>Arrays.fill</code> , <code>Arrays.copyOf</code> . Strings: immutability and why it matters; <code>charAt</code> , <code>substring</code> , <code>indexOf</code> , <code>contains</code> , <code>split</code> , <code>trim</code> , <code>toCharArray</code> , <code>replace</code> ; <code>equals</code> versus <code>==</code> and the string pool; <code>StringBuilder</code> and why concatenation in a loop is a performance bug.
DRILLS · Array manipulation and string handling written by hand <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write your own methods without using library shortcuts: <code>findMax</code>, <code>findMin</code>, <code>sumArray</code>, <code>averageArray</code>, <code>reverseArrayInPlace</code>. 2. Write <code>containsValue(int[] arr, int target)</code> with a linear scan, then write <code>countOccurrences</code>. 3. Write <code>rotateArray(int[] arr, int k)</code> that rotates an array right by <code>k</code> positions in place. 4. Build a 2D int array representing a 4x4 matrix. Write methods to print it, sum each row, sum each column, and sum the main diagonal. 5. Write your own <code>StringUtils</code> class implementing by hand: <code>reverse</code>, <code>isPalindrome</code> (ignoring case and spaces), <code>countVowels</code>, <code>countWords</code>, <code>capitalizeEachWord</code>, <code>removeDuplicateChars</code>. 6. Demonstrate the <code>equals</code> versus <code>==</code> trap: create two Strings with the literal syntax and two with 'new String', compare all pairs both ways, and write a comment explaining every result. 7. Time a loop that concatenates 50,000 strings with <code>+</code> against the same loop using <code>StringBuilder</code>. Print both durations with <code>System.nanoTime</code>. 8. Write a method that takes a sentence and returns the longest word, and another that returns a word-frequency HashMap.	
120 min Build	Add a <code>Day03</code> package with all drills committed. Then write a small text-analysis program: read a paragraph, report character count, word count, sentence count, the five most frequent words, and the longest word.

✓ **Win of the day: You implemented a string library by hand and proved a real performance difference with your own measurements.**

Day 4 — Monday, Sep 7

Recursion and classes

60 min Algorithms	Pattern: Arrays & Hashing (part 3) — sequences and set lookups A HashSet gives $O(1)$ membership testing, which converts many 'is this element present' scans into a single pass. Solve in this order: 1. Longest Consecutive Sequence (medium) — HashSet, only start counting at sequence starts 2. Encode and Decode Strings (medium) — length-prefixed encoding design 3. Valid Sudoku (medium) — three sets of hash sets
150 min Learn	Recursion: base case, recursive case, the call stack, StackOverflowError, and when recursion beats iteration. Then classes and objects: fields, constructors, constructor overloading, the this keyword, getters and setters, access modifiers (private, default, protected, public), static fields and static methods, and the object lifecycle.
DRILLS · Recursion and your first encapsulated classes <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write recursive and iterative versions of: factorial, fibonacci, sum of digits, power(base, exp), and reverse a string. Time fibonacci(40) both ways and note the difference. 2. Write a recursive binary search on a sorted array, and a recursive method that counts occurrences of a character in a string. 3. Write a recursive method to print all elements of a nested integer array structure. 4. Deliberately write a recursion with no base case, run it, and observe the StackOverflowError. Write a comment explaining the stack. 5. Create a BankAccount class with private balance, accountNumber, ownerName; a constructor; deposit and withdraw methods that reject invalid amounts; and getBalance. Do not allow direct field access. 6. Add a static field accountCount to BankAccount that increments in the constructor, and a static method getAccountCount. Create five accounts and print the count. 7. Create a Book class with id, title, author, isbn, available; two overloaded constructors (one with all fields, one with defaults); full encapsulation; and a toString override. 8. Write a Member class with id, name, email and demonstrate creating an array of five Members and printing them all.	
120 min Build	Start the Library Management System in a new package. Create the Book and Member classes properly encapsulated. Write a Library class holding a Book array with addBook and listBooks methods, and a main that demonstrates it.

✓ **Win of the day: Your two-week project exists, and you designed its domain classes with proper encapsulation yourself.**

Day 5 — Tuesday, Sep 8

Inheritance and polymorphism

60 min Algorithms	Pattern: Two Pointers (part 1) — converging pointers on sorted data When data is sorted or symmetric, put one pointer at each end and move them toward each other. Each comparison eliminates a whole side of the search space. Solve in this order: 1. Valid Palindrome (easy) — pointers from both ends 2. Two Sum II Input Array Is Sorted (medium) — move left up or right down based on the sum 3. 3Sum (medium) — fix one element, two-pointer the rest
150 min Learn	Inheritance with extends; the super keyword for both fields and constructors; constructor chaining order; method overriding rules; @Override and why it matters; overriding versus overloading; the Object class and its methods; toString, equals, and hashCode and the contract between the last two; polymorphism and dynamic dispatch; the instanceof operator and pattern matching; upcasting and downcasting; final classes and methods.
DRILLS · Inheritance, polymorphism, and the equals/hashCode contract <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Build a Vehicle base class (make, model, year, speed) with a describe method. Create Car, Motorcycle, and Truck subclasses, each overriding describe with a distinct output. 2. In each subclass constructor, call super explicitly and add at least one subclass-specific field. Add a print statement in every constructor and observe the exact order they execute. 3. Create a Vehicle array holding one of each subclass, loop over it calling describe, and confirm the subclass version runs each time. Write a comment explaining dynamic dispatch. 4. Write a method that takes a Vehicle parameter and uses instanceof to perform a type-specific action for each subclass, and then rewrite it using pattern-matching instanceof. 5. Override equals and hashCode on Book so two Books with the same ISBN are considered equal. Prove it by putting duplicates in a HashSet and showing only one survives. 6. Deliberately break the contract: override equals but NOT hashCode, put two 'equal' objects in a HashSet, and observe both being stored. Write a comment explaining why. 7. Create a class hierarchy for Employee with Manager and Developer subclasses, each with its own calculatePay implementation. Loop over a mixed array and print total payroll. 8. Attempt to override a method marked final and observe the compiler error. Attempt to extend a final class and observe the same.	
120 min Build	Refactor the Library: create an abstract-in-spirit LibraryItem parent class (you will make it truly abstract tomorrow) and make Book, Magazine, and DVD extend it. Store mixed types in a LibraryItem array and print them all polymorphically.

✓ **Win of the day: Your library handles three item types through one abstraction, and you have proven the equals/hashCode contract by breaking it.**

Day 6 — Wednesday, Sep 9

Abstract classes, interfaces, and enums

60 min Algorithms	Pattern: Two Pointers (part 2) — in-place modification and partitioning A read pointer and a write pointer let you rearrange an array in place in one pass, with no extra memory. Solve in this order: 1. Remove Duplicates from Sorted Array (easy) — slow/fast write pointer 2. Move Zeroes (easy) — same technique 3. Container With Most Water (medium) — greedy converging pointers
150 min Learn	Abstract classes: abstract methods, why you cannot instantiate them, and when to use one. Interfaces: multiple implementation, default methods, static methods, constants, and functional interfaces. The decision rule for interface versus abstract class — a guaranteed interview question. Enums: simple enums, enums with fields and constructors, enums with abstract methods, and using enums in switch statements.
DRILLS · Abstraction — interfaces, abstract classes, and enums <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Convert <code>LibraryItem</code> into a genuine abstract class with an abstract method <code>calculateLateFee(int daysLate)</code> implemented differently in <code>Book</code>, <code>Magazine</code>, and <code>DVD</code>. 2. Try to instantiate <code>LibraryItem</code> directly and observe the compiler error. Write a comment explaining why abstract classes exist. 3. Define a <code>Borrowable</code> interface with <code>borrow</code>, <code>returnItem</code>, and <code>isAvailable</code>. Implement it on <code>Book</code> and <code>DVD</code> but not <code>Magazine</code>, and write a method that accepts <code>Borrowable</code> and works with either. 4. Define a second interface <code>Searchable</code> with a default method <code>matches(String query)</code>. Implement both interfaces on <code>Book</code> and demonstrate that Java allows multiple interface implementation but not multiple class inheritance. 5. Write a one-paragraph note in your log answering, in your own words: when do you choose an interface over an abstract class? You will be asked this in an interview. 6. Create an <code>ItemStatus</code> enum (<code>AVAILABLE</code>, <code>BORROWED</code>, <code>RESERVED</code>, <code>LOST</code>) and use it in a switch expression that returns a human-readable message for each. 7. Create a <code>Genre</code> enum with a constructor field (a display name in French, since you are targeting Montreal) and a getter. Loop over <code>Genre.values</code> and print each. 8. Create an <code>Operation</code> enum with an abstract <code>apply(double a, double b)</code> method implemented per constant (<code>ADD</code>, <code>SUBTRACT</code>, <code>MULTIPLY</code>, <code>DIVIDE</code>), then use it to rebuild your Day 2 calculator without a switch.	
120 min Build	Refactor the Library fully onto abstractions: abstract <code>LibraryItem</code> , <code>Borrowable</code> and <code>Searchable</code> interfaces, <code>ItemStatus</code> and <code>Genre</code> enums. Every item type must implement late-fee logic differently. Verify everything still runs.

✓ **Win of the day: Your project is now built on interfaces and abstract types, and you can defend the design choice out loud.**

Day 7 — Thursday, Sep 10

Exceptions and file input/output

60 min Algorithms	Pattern: Arrays & Hashing (part 4) — the optimal HashMap solution, revisited Return to Day 1's brute-force Two Sum and see the transformation: one pass, storing what you have seen in a map, looking up the complement. This is the moment the pattern clicks. Solve in this order: 1. Two Sum (easy) — now with a HashMap in one pass, under 10 minutes 2. Subarray Sum Equals K (medium) — prefix sums stored in a map 3. Longest Common Prefix (easy) — a palate cleanser
150 min Learn	The exception hierarchy: Throwable, Error, Exception, RuntimeException. Checked versus unchecked and when each is appropriate. try, catch, multi-catch, finally, and try-with-resources. throw versus throws. Creating custom exception classes. Exception chaining with a cause. Reading a stack trace properly. Then file I/O: Files.readAllLines, Files.write, BufferedReader and BufferedWriter, and the Path API.
DRILLS · Exception design and file persistence <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write code that triggers and catches each of these deliberately: ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, NumberFormatException, ClassCastException. Print a distinct message for each. 2. Write a method that demonstrates finally executing even when an exception is thrown, and even when the try block returns. 3. Create a custom checked exception BookNotFoundException and a custom unchecked exception InvalidIsbnException. Use each in the appropriate place in your Library and explain in a comment why you chose checked or unchecked for each. 4. Write a method that catches a low-level exception and rethrows it wrapped in your own exception with the original as the cause. Print the full stack trace and identify both layers in it. 5. Rewrite a file-reading method three ways: with try/catch/finally closing manually, with try-with-resources, and observe how much shorter the second is. 6. Write saveToFile(List) and loadFromFile() for your Library using a simple delimited text format. Handle a missing file gracefully rather than crashing. 7. Deliberately corrupt your saved file (delete a field from one line), run the loader, and make it skip the bad line with a warning instead of failing entirely. 8. Write a method that validates an ISBN and throws InvalidIsbnException with a message naming the specific problem.	
120 min Build	Add full error handling to the Library. Borrowing an unavailable item, returning something never borrowed, and looking up a missing ISBN must all throw meaningful custom exceptions caught and reported cleanly in the menu loop. Add persistence so the catalogue survives restart.

✓ **Win of the day: Your application fails gracefully with meaningful errors, and its data survives a restart.**

Day 8 — Friday, Sep 11

Collections part one — Lists

60 min Algorithms	Pattern: Sliding Window (part 1) — fixed and variable windows Maintain a window over the array or string. Expand the right edge to include more, shrink the left edge when the window becomes invalid. Every element enters and leaves once, so it is $O(n)$. Solve in this order: 1. Best Time to Buy and Sell Stock (easy) — a window of one buy and one sell 2. Longest Substring Without Repeating Characters (medium) — shrink left when a duplicate appears 3. Longest Repeating Character Replacement (medium) — window valid while replacements needed is within k
150 min Learn	The Collections framework hierarchy. The List interface; ArrayList backed by an array versus LinkedList backed by nodes, and the performance consequences of each operation; capacity and resizing; add, remove, get, set, contains, indexOf, subList; iterating with for-each versus Iterator; ConcurrentModificationException and how to remove safely; List.of and immutability; Arrays.asList and its trap.
DRILLS · Mastering Lists, and understanding ArrayList internals <i>Goal: write every one of these yourself, by hand, no copy-paste.</i> 1. Create an ArrayList of Strings, add ten items, then remove every item containing the letter 'a' using an Iterator. Then attempt the same with a for-each loop and observe ConcurrentModificationException. 2. Benchmark ArrayList versus LinkedList: time 100,000 add-at-end operations, then 10,000 insert-at-index-0 operations on each. Print the four numbers and write a comment explaining the results. 3. Write your own MyArrayList class backed by an Object array, supporting add, get, remove, size, and automatic resizing when full. This teaches you what ArrayList actually does. 4. Write a method that takes a List and returns a new List with duplicates removed, preserving order, without using a Set. 5. Create a List with List.of, then attempt to add to it. Observe UnsupportedOperationException and write a comment on immutability. 6. Write methods that merge two sorted Lists into one sorted List, and that partition a List into two Lists based on a condition. 7. Replace every array in the Library System with an ArrayList and add addItem, removeItem, findByTitle, and listAvailable methods.	
120 min Build	Complete the ArrayList migration of the Library. It should now grow without limit. Add a borrow-history List to Member. Commit.

✓ **Win of the day: You built your own ArrayList from scratch, so you know exactly what the real one is doing.**

Day 9 — Saturday, Sep 12

Collections part two — Maps and Sets

60 min Algorithms	Pattern: Sliding Window (part 2) — windows with a frequency map Combine the window with a HashMap counting what is inside it. This handles anagram and permutation problems where you need the window's contents, not just its size. Solve in this order: 1. Permutation in String (medium) — fixed window with character counts 2. Minimum Window Substring (hard) — attempt seriously for 30 minutes even if you do not finish 3. Sliding Window Maximum (hard) — read the deque solution and re-implement from memory
150 min Learn	The Map interface; HashMap internals — hashing, buckets, collision handling, load factor, and resizing; why a good hashCode matters; TreeMap for sorted keys and its $O(\log n)$ trade-off; LinkedHashMap for insertion order; getOrDefault, putIfAbsent, computeIfAbsent, merge; iterating entrySet, keySet, values. The Set interface; HashSet, TreeSet, LinkedHashSet. Choosing the right collection for a given requirement.
DRILLS · Maps, Sets, and why hashCode quality matters <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Build a word-frequency counter for a paragraph three ways: with get and put, with getOrDefault, and with merge. Compare readability. 2. Use computeIfAbsent to build a Map<String, List> grouping words by first letter. 3. Create a class with a deliberately terrible hashCode that always returns 1. Put 10,000 objects in a HashMap and time lookups against a proper hashCode implementation. Print both timings. 4. Demonstrate the difference between HashMap, LinkedHashMap, and TreeMap by inserting the same five keys in random order into each and printing the iteration order of all three. 5. Write a method that inverts a Map (values become keys) and handles duplicate values by mapping to a List. 6. Use a HashSet to find the intersection, union, and difference of two Lists of integers. 7. Add a HashMap index to your Library for instant ISBN lookup, and a HashMap<String, List> tracking borrowed items. 8. Write a method findMostBorrowedAuthor using a frequency map over the borrow history.	
120 min Build	Wire the HashMap indexes into the Library so lookups no longer loop. Add a HashSet enforcing unique ISBNs and rejecting duplicates with a clear exception. Verify with tests in main.

✓ **Win of the day: You proved with your own benchmark why hashCode quality matters, and your library lookups are now instant.**

Day 10 — Sunday, Sep 13

Generics, Comparable, and sorting

60 min Algorithms	Pattern: Stack — matching, nesting, and monotonic stacks A stack tracks 'what am I still waiting to close'. For monotonic stack problems, you keep the stack sorted and pop whenever the new element breaks the order — that pop is the answer for the popped element. Solve in this order: 1. Valid Parentheses (easy) — classic matching 2. Min Stack (medium) — a second stack tracking minimums 3. Evaluate Reverse Polish Notation (medium) 4. Daily Temperatures (medium) — first monotonic stack
150 min Learn	Why generics exist: the pre-generics casting problem and type erasure. Generic classes, generic methods, multiple type parameters, bounded type parameters with extends, wildcards and the difference between an unbounded, upper-bounded, and lower-bounded wildcard. Then Comparable and compareTo for natural ordering; Comparator for alternative orderings; Collections.sort and List.sort; comparator chaining with comparing and thenComparing and reversed.
DRILLS · Generics, Comparable, and custom sorting <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write a generic Box class with set, get, and isEmpty. Instantiate it with three different types. 2. Write a generic method void printAll(List list) and a generic method > T findMax(List list). Test findMax on Integers, Strings, and your Book class. 3. Write a generic Repository class with add, remove, findAll, and findById using a Function to extract the id. This is the pattern Spring Data will hand you in Week 4. 4. Make LibraryItem implement Comparable ordered by title, then sort a List of items and print the result. 5. Write four separate Comparators for Book: by author, by year descending, by title length, and by availability then title. Sort and print with each. 6. Chain comparators: sort books by genre, then by author, then by title, using comparing and thenComparing in one statement. 7. Demonstrate the wildcard problem: write a method taking List, try to pass a List, observe the compiler error, then fix it with a wildcard and explain why in a comment.	
120 min Build	Add sorting throughout the Library: sort the catalogue by any field on demand from the menu. Refactor storage to use your generic Repository.

✓ **Win of the day: You wrote a generic repository and four comparators — the exact abstractions professional Java code is built from.**

Day 11 — Monday, Sep 14

Lambdas and the Stream API

60 min Algorithms	Pattern: Binary Search (part 1) — searching a sorted array Halve the search space every step. The hard part is never the idea, it is getting the boundary conditions right — which is why you write the template until it is automatic. Solve in this order: 1. Binary Search (easy) — write the template from memory five times 2. Search Insert Position (easy) 3. Search a 2D Matrix (medium) — treat the matrix as one flat sorted array 4. Find Minimum in Rotated Sorted Array (medium)
150 min Learn	Functional interfaces: Function, BiFunction, Predicate, Consumer, Supplier, UnaryOperator. Lambda syntax and variable capture (effectively final). Method references in all four forms. The Stream API: creating streams; intermediate operations filter, map, flatMap, distinct, sorted, limit, skip, peek; terminal operations collect, forEach, reduce, count, anyMatch, allMatch, findFirst; the Collectors class including toList, toSet, joining, groupingBy, partitioningBy, counting, summingInt, averagingDouble. Lazy evaluation and why streams do nothing until a terminal operation runs.
DRILLS · Lambdas and the Stream API <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write the same filtering task three ways: a classic for loop, an enhanced for loop, and a stream. Compare line counts. 2. Write one lambda for each of Function, Predicate, Consumer, and Supplier, and call each. 3. Convert five lambdas into method references where possible; identify one that cannot be converted and explain why. 4. From a List of Books: filter to available ones, map to titles, sort alphabetically, and collect to a List — in one stream chain. 5. Use groupingBy to build a Map of genre to List of books, then groupingBy with counting to get genre to count. 6. Use partitioningBy to split books into available and unavailable. 7. Use reduce to compute the total late fees across a list of overdue items, and use mapToInt with sum to do the same thing more simply. 8. Add a peek to a stream chain with no terminal operation, run it, observe that nothing prints, then add collect and observe it run. Write a comment on lazy evaluation. 9. Rewrite four of your existing Library methods to use streams instead of manual loops.	
120 min Build	Refactor Library search, filter, and reporting onto streams. Add three report methods: most borrowed author, item count by genre, and average days borrowed.

✓ **Win of the day: Your code went from verbose loops to declarative stream pipelines — a visible, immediate level-up in code quality.**

Day 12 — Tuesday, Sep 15

Optional, records, and modern Java

60 min Algorithm s	Pattern: Binary Search (part 2) — binary search on the answer space The array is not always what you search. When a question asks for a minimum or maximum value satisfying a condition that is monotonic, binary search over the possible answers instead. Solve in this order: 1. Koko Eating Bananas (medium) — search over eating speeds 2. Search in Rotated Sorted Array (medium) 3. Time Based Key-Value Store (medium) 4. Median of Two Sorted Arrays (hard) — attempt for 30 minutes, then study the solution
150 min Learn	Null and the NullPointerException problem. Optional: creating with of, ofNullable, empty; consuming with isPresent, ifPresent, orElse, orElseGet, orElseThrow, map, filter. When Optional is appropriate (return types) and when it is not (fields and parameters). Records: compact syntax, automatic equals/hashCode/toString, immutability, compact constructors for validation. var and local type inference. Switch expressions with arrow syntax and yield. Text blocks. Sealed classes at a conceptual level.
DRILLS · Optional, records, and modern Java syntax <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write a method that returns null on failure, call it, and cause a NullPointerException. Then rewrite it returning Optional and handle the empty case three different ways. 2. Convert every findById-style method in your Library to return Optional and update all callers with orElseThrow using your custom exception. 3. Write a chain: Optional of a Member, map to their borrowed list, filter to non-empty, map to size, orElse zero — all in one expression. 4. Convert a simple data-only class into a record and observe what you no longer have to write. Print the auto-generated toString. 5. Add a compact constructor to a record that validates a field and throws on invalid input. Prove it rejects bad data. 6. Attempt to modify a record field after construction and observe the compiler error. 7. Rewrite three older switch statements as switch expressions with arrow syntax, one of them using yield. 8. Write a text block containing a formatted multi-line receipt for a borrow transaction. 9. Do a full naming and structure pass over the whole Library codebase: better names, shorter methods, tightened access modifiers, no dead code.	
120 min Build	Modernize the entire codebase: Optional returns, records where appropriate, enums finalized, switch expressions, and a full clean-up pass. All existing functionality must still work.

✓ **Win of the day: Your codebase now reads like modern professional Java, not beginner Java.**

Day 13 — Wednesday, Sep 16

Maven and JUnit 5

60 min Algorithms	Pattern: Linked List (part 1) — pointer manipulation and dummy heads Linked list problems are about carefully rewiring next pointers without losing the rest of the list. A dummy head node removes almost all edge-case handling. Solve in this order: 1. Reverse Linked List (easy) — write iterative and recursive versions 2. Merge Two Sorted Lists (easy) — use a dummy head 3. Linked List Cycle (easy) — fast and slow pointers 4. Remove Nth Node From End of List (medium) — two pointers offset by n
150 min Learn	Maven: the problem it solves, the standard directory layout, pom.xml structure, groupId/artifactId/version coordinates, dependencies and scopes (compile, test, provided), plugins, the build lifecycle phases (validate, compile, test, package, verify, install), and the commands you will use daily. Then JUnit 5: @Test, the assertion methods, assertThrows, @BeforeEach and @AfterEach, @DisplayName, @ParameterizedTest with @ValueSource and @CsvSource, @Nested for grouping, and test naming conventions.
DRILLS · Maven builds and your first JUnit test suite <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Convert the Library System into a Maven project with the correct src/main/java and src/test/java layout. Get mvn clean install to succeed. 2. Add a dependency (for example Apache Commons Lang) with test scope, use one method from it in a test, and confirm it is unavailable in main code. 3. Run each of mvn compile, mvn test, mvn package separately and note what each produces in the target directory. 4. Write your first five tests for the Book class: construction, toString, equals, hashCode consistency, and validation rejection. 5. Write tests for borrowing: a successful borrow, borrowing an unavailable item (assertThrows), returning a borrowed item, and returning something never borrowed. 6. Write a @ParameterizedTest with @CsvSource covering at least six ISBN validation cases, valid and invalid. 7. Use @BeforeEach to set up a fresh Library with three books before every test, eliminating duplicated setup. 8. Group your tests with @Nested into BorrowingTests, SearchTests, and ValidationTests, each with a @DisplayName. 9. Deliberately break one method in your main code, run the suite, watch the test fail, then fix it. This is the feedback loop you will live in professionally.	
120 min Build	Build the full test suite: 20 or more tests covering borrowing, returning, searching, sorting, validation, and every custom exception. Everything green with one Maven command.

✓ **Win of the day: Your project builds and tests with a single command, and a regression tomorrow would be caught automatically.**

Day 14 — Thursday, Sep 17

Consolidation and first presentation

60 min Algorithms	Pattern: Review — all patterns from Week 1 and 2 Retrieval practice beats re-reading. Solve from a blank file with a timer; anything you cannot finish in 25 minutes goes on your revisit list for Day 21. Solve in this order: 1. Contains Duplicate — 5 min target 2. Valid Palindrome — 10 min target 3. Longest Substring Without Repeating Characters — 20 min target 4. Valid Parentheses — 10 min target 5. Binary Search — 10 min target 6. Reverse Linked List — 10 min target
150 min Learn	No new material. Re-read your log from Days 1 to 13 and re-derive anything that still feels shaky. Then read your entire Library codebase top to bottom as a hostile reviewer would, writing down every weakness you find.
DRILLS · Retrieval practice — proving Weeks 1-2 knowledge is yours <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. From a blank file, with no reference: write a class with a constructor, a private field, a getter, and a toString. 2. From a blank file: write an interface, an abstract class implementing it, and a concrete subclass. 3. From a blank file: write a method that groups a List of objects into a Map using streams. 4. From a blank file: write a custom exception and a method that throws it under a condition. 5. From a blank file: write a JUnit test with a @BeforeEach and an assertThrows. 6. Anything above that you could not write unaided goes into your log as a gap, and you spend the Learn block closing it. 7. Complete the console menu so every Library feature is reachable and every error path handled. 8. Write the README: what the project does, its architecture, the OOP concepts demonstrated, and how to run it.	
120 min Build	Final polish and push. Then record yourself giving a three-minute verbal walkthrough of the project: what it does, how it is structured, and one design decision you made and why. Watch it back once.

✓ **Win of the day: Fourteen days ago you had never written Java. You now have a tested, Maven-built application you can explain out loud.**